Architecture et Fonctionnement des Systèmes d'Exploitation : Du Programme au Processus & Signaux POSIX

Thème principal : Gestion des processus, multitâche préemptif, cycle de vie, arborescence PID/PPID et communication inter-processus par signaux POSIX

Synthèse exécutive : Ce cours fondamental de R1.04 détaille la transition essentielle entre un programme binaire statique (stocké sur disque) et un processus dynamique (chargé en mémoire RAM). Il explore la mécanique du multitâche préemptif géré par le noyau Linux, la modélisation du cycle de vie des processus à travers trois états fondamentaux (Élu, Prêt, Bloqué), ainsi que l'arborescence généalogique issue du processus initial systemd (PID 1) via les appels système fork() et exec(). Enfin, le cours explicite la communication inter-processus asynchrone à l'aide des signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Pour qu'un ordinateur puisse exécuter un code, le système d'exploitation doit faire la distinction fondamentale entre un programme et un processus. Un programme est un fichier binaire inerte enregistré sur un support de stockage secondaire (SSD/HDD). Lorsqu'un utilisateur ou le système demande son exécution, le noyau (kernel) alloue un espace en mémoire vive (RAM) et crée une structure interne appelée le PCB (Process Control Block) : le programme devient alors un processus actif. Afin de faire fonctionner des dizaines d'applications en même temps sur un processeur, le noyau s'appuie sur le multitâche préemptif. L'ordonnanceur (scheduler) attribue de très courtes tranches de temps d'exécution (quanta de temps) à chaque processus. Le cycle de vie d'un processus s'articule autour de trois états fondamentaux : 1. Élu (Running) : Le processus est pris en charge par le CPU et exécute ses instructions. 2. Prêt (Ready) : Le processus est complètement chargé en RAM et attend que l'ordonnanceur lui attribue un quantum CPU. 3. Bloqué (Waiting/Blocked) : Le processus est mis en pause car il attend un événement externe (lecture disque, réception d'un paquet réseau, saisie clavier). Dès que l'événement survient, le processus ne repasse JAMAIS directement en état Élu : il bascule en état Prêt pour réintégrer la file d'attente de l'ordonnanceur. Tous les processus s'organisent selon un arbre généalogique strict. Au démarrage de la machine, le noyau crée le tout premier processus utilisateur nommé systemd (ou init), portant le PID 1 (Process Identifier). Tout nouveau processus est créé par duplication de son parent (PPID - Parent Process Identifier) via l'appel système fork(), puis le fils remplace son code exécutable par celui du nouveau programme cible via l'appel système exec().

- • Programme vs Processus : Un programme est un fichier binaire passif sur disque ; un processus est son instance dynamique en mémoire RAM avec ressources allouées.
- • Multitâche préemptif & Ordonnanceur : Découpage du temps CPU en quanta attribués dynamiquement aux processus éligibles.
- • Bloc de Contrôle de Processus (PCB) : Structure de données noyau conservant l'état CPU, les registres, la carte mémoire et les identifiants.
- • Cycle de vie à 3 états : Élu (Running - occupe le CPU), Prêt (Ready - attend son tour CPU en RAM), Bloqué (Waiting - en attente d'une I/O).
- • Arborescence et Création : systemd/init (PID 1) est le père de tous les processus. La création s'effectue par fork() (duplication du parent) suivi de exec() (remplacement du code binaire).

Cas concret enseignant : *L'analogie du livre de recettes : Le programme sur disque dur est comparé à un livre de recettes rangé sagement dans une bibliothèque (statique, passif, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier qui prépare activement la recette dans sa cuisine (dynamique en RAM, occupant du plan de travail, consommant de l'énergie CPU et manipulant des ustensiles/descripteurs de fichiers).*

2. Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux POSIX constituent le mécanisme de communication inter-processus (IPC) le plus simple et universel sous les systèmes Unix/Linux. Un signal est une interruption logicielle envoyée de manière asynchrone à un processus pour lui notifier un événement d'arrière-plan ou une instruction d'arrêt. À la réception d'un signal, un programme dispose de trois choix d'implémentation : 1. Laisser le système exécuter l'action par défaut (généralement la fermeture du programme). 2. Ignorer sciemment le signal. 3. Capturer le signal (catch) en définissant un gestionnaire personnalisé (Signal Handler) pour effectuer des traitements préalables (sauvegarder les fichiers ouverts, libérer la mémoire). Cependant, il existe une règle absolue pour garantir le contrôle du système par l'administrateur : les signaux SIGKILL (Signal 9) et SIGSTOP (Signal 19) sont gérés directement par le noyau Linux. Ils ne peuvent JAMAIS être interceptés, masqués ou ignorés par aucun programme, garantissant ainsi qu'un processus bloqué ou malveillant puisse toujours être détruit.

- Signal POSIX : Notification ou interruption logicielle asynchrone envoyée à un processus pour modifier son comportement.
- Traitement d'un signal : Un processus peut exécuter l'action par défaut, ignorer le signal, ou l'intercepter via un gestionnaire de signal (Signal Handler).
- Signaux critiques : SIGINT (2 - interruption interactive), SIGTERM (15 - demande de fermeture propre), SIGKILL (9 - destruction forcée inconditionnelle par le noyau).
- Contrôle des tâches (Job Control) : SIGSTOP (19 - suspension du processus) et SIGCONT (18 - reprise de l'exécution).

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble : SIGTERM (-15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de ranger vos affaires et d'évacuer (vous avez le temps de sauvegarder vos documents et fermer la porte à clé). SIGKILL (-9) est une intervention d'urgence de la brigade de sécurité qui défonce la porte et vous dégage sur-le-champ sans vous laisser toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne correctement ?
Réponse du professeur : Sans le tiret (-), l'utilitaire `kill` interprète '9' comme étant un numéro de processus (PID) et non un numéro de signal. La commande cherche donc à envoyer le signal par défaut (SIGTERM / 15) aux processus ayant pour PID 9 puis 1234. Pour spécifier un signal par son numéro ou son nom, il faut impérativement préfixer le paramètre par un tiret (ex: `kill -9 1234` ou `kill -SIGKILL 1234`).

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire avec la commande kill ?
Réponse du professeur : Un processus Zombie (marqué état 'Z' dans la commande ps) est un processus enfant qui a terminé son exécution mais dont le parent n'a pas encore lu le code de sortie (via l'appel système wait()). Le zombie ne consomme plus ni CPU ni RAM, c'est juste une entrée restante dans la table des processus. On NE PEUT PAS le détruire avec `kill -9` car il est DÉJÀ mort ! Pour le nettoyer, il faut forcer le processus parent à lire son statut, ou tuer le processus parent : le zombie est alors adopté par systemd (PID 1) qui nettoie immédiatement l'entrée.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un étudiant saisit la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez ce qui se passe lors de l'envoi du signal -15 et décrivez le résultat de la vérification ultérieure par la commande ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' placé en fin de ligne indique au shell d'exécuter la commande 'sleep 600' en arrière-plan (background). Le shell affiche l'identifiant du job (ex: [1]) ainsi que le PID attribué par le noyau (ex: 4521) et rend immédiatement la main à l'utilisateur. Étape 2 : La commande 'ps -ef | grep sleep' parcourt la table de tous les processus système et filtre la ligne correspondant à l'exécutable 'sleep'. Cela permet de repérer son PID (4521)

ainsi que son PPID (qui est le PID du shell Bash hôte). Étape 3 : La commande 'kill -15 4521' envoie le signal SIGTERM (Signal 15) au processus 4521. N'ayant pas de gestionnaire personnalisé défini pour intercepter ce signal, l'application applique l'action par défaut du système : l'arrêt propre et la libération des ressources.

Résultat : Le processus 'sleep 600' est immédiatement terminé de façon propre. Une nouvelle exécution de la commande 'ps -ef | grep sleep' confirme sa disparition définitive de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : Le premier processus utilisateur est systemd (ou init sur les anciennes distributions) et porte obligatoirement le PID 1.

Q : Quel signal POSIX est généré et transmis au processus s'exécutant au premier plan lorsqu'un utilisateur presse la combinaison de touches 'Ctrl + C' ?

R : La combinaison 'Ctrl + C' émet le signal SIGINT (Signal 2), qui demande l'interruption interactive du processus.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté, masqué ou ignoré par un programme informatique ?

R : Car il est géré directement et exclusivement par le noyau (kernel) du système d'exploitation pour garantir qu'un administrateur puisse toujours stopper un processus malveillant ou bloqué.